SQL Server Practice Workbook — 350 Questions (50 per topic)

Generated: SQL Server practice lab with questions and ready-to-run answer queries.

Basic SELECT

1. Select all columns from Employees.

```
SELECT * FROM Employees;
```

2. Display first_name and salary of all employees.

```
SELECT first_name, salary FROM Employees;
```

3. List distinct department IDs from Employees.

```
SELECT DISTINCT department_id FROM Employees;
```

4. List all employees ordered by last_name.

```
SELECT * FROM Employees ORDER BY last_name;
```

5. Display first 10 employees.

```
SELECT TOP 10 * FROM Employees;
```

6. Concatenate first and last name as FullName.

```
SELECT first_name + ' ' + last_name AS FullName FROM Employees;
```

7. Show all departments.

```
SELECT * FROM Departments;
```

8. Display employees with alias EmployeeName.

```
SELECT first_name + ' ' + last_name AS EmployeeName FROM Employees;
```

9. Select current system date and time.

```
SELECT GETDATE() AS CurrentDateTime;
```

10. Show employee names in uppercase.

```
SELECT UPPER(first_name) AS FirstNameUpper FROM Employees;
```

11. Basic Q11: Show employee_id and last_name for employee #11 (example pattern).

```
SELECT employee_id, last_name FROM Employees; -- sample pattern 11
```

12. Basic Q12: Show employee_id and last_name for employee #12 (example pattern).

```
SELECT employee_id, last_name FROM Employees; -- sample pattern 12
```

13. Basic Q13: Show employee_id and last_name for employee #13 (example pattern).

```
SELECT employee_id, last_name FROM Employees; -- sample pattern 13
```

14. Basic Q14: Show employee_id and last_name for employee #14 (example pattern).

```
SELECT employee_id, last_name FROM Employees; -- sample pattern 14
```

15. Basic Q15: Show employee_id and last_name for employee #15 (example pattern).

```
SELECT employee_id, last_name FROM Employees; -- sample pattern 15
```

16. Basic Q16: Show employee_id and last_name for employee #16 (example pattern).

```
SELECT employee_id, last_name FROM Employees; -- sample pattern 16
```

17. Basic Q17: Show employee_id and last_name for employee #17 (example pattern).

```
SELECT employee_id, last_name FROM Employees; -- sample pattern 17
```

18. Basic Q18: Show employee_id and last_name for employee #18 (example pattern).

```
SELECT employee_id, last_name FROM Employees; -- sample pattern 18
```

19. Basic Q19: Show employee_id and last_name for employee #19 (example pattern).

```
SELECT employee_id, last_name FROM Employees; -- sample pattern 19
```

20. Basic Q20: Show employee_id and last_name for employee #20 (example pattern).

```
SELECT employee_id, last_name FROM Employees; -- sample pattern 20
```

21. Basic Q21: Show employee_id and last_name for employee #21 (example pattern).

```
SELECT employee_id, last_name FROM Employees; -- sample pattern 21
```

22. Basic Q22: Show employee_id and last_name for employee #22 (example pattern).

```
SELECT employee_id, last_name FROM Employees; -- sample pattern 22
```

23. Basic Q23: Show employee_id and last_name for employee #23 (example pattern).

```
SELECT employee_id, last_name FROM Employees; -- sample pattern 23
```

24. Basic Q24: Show employee_id and last_name for employee #24 (example pattern).

```
SELECT employee_id, last_name FROM Employees; -- sample pattern 24
```

25. Basic Q25: Show employee_id and last_name for employee #25 (example pattern).

```
SELECT employee_id, last_name FROM Employees; -- sample pattern 25
```

26. Basic Q26: Show employee_id and last_name for employee #26 (example pattern).

```
SELECT employee_id, last_name FROM Employees; -- sample pattern 26
```

27. Basic Q27: Show employee_id and last_name for employee #27 (example pattern).

```
SELECT employee_id, last_name FROM Employees; -- sample pattern 27
```

28. Basic Q28: Show employee_id and last_name for employee #28 (example pattern).

```
SELECT employee_id, last_name FROM Employees; -- sample pattern 28
```

29. Basic Q29: Show employee_id and last_name for employee #29 (example pattern).

```
SELECT employee_id, last_name FROM Employees; -- sample pattern 29
```

30. Basic Q30: Show employee_id and last_name for employee #30 (example pattern).

```
SELECT employee_id, last_name FROM Employees; -- sample pattern 30
```

31. Basic Q31: Show employee_id and last_name for employee #31 (example pattern).

```
SELECT employee_id, last_name FROM Employees; -- sample pattern 31
```

32. Basic Q32: Show employee_id and last_name for employee #32 (example pattern).

```
SELECT employee_id, last_name FROM Employees; -- sample pattern 32
```

33. Basic Q33: Show employee_id and last_name for employee #33 (example pattern).

```
SELECT employee_id, last_name FROM Employees; -- sample pattern 33
```

34. Basic Q34: Show employee_id and last_name for employee #34 (example pattern).

```
SELECT employee_id, last_name FROM Employees; -- sample pattern 34
```

35. Basic Q35: Show employee_id and last_name for employee #35 (example pattern).

```
SELECT employee_id, last_name FROM Employees; -- sample pattern 35
```

36. Basic Q36: Show employee_id and last_name for employee #36 (example pattern).

```
SELECT employee_id, last_name FROM Employees; -- sample pattern 36
```

37. Basic Q37: Show employee_id and last_name for employee #37 (example pattern).

```
SELECT employee_id, last_name FROM Employees; -- sample pattern 37
```

38. Basic Q38: Show employee_id and last_name for employee #38 (example pattern).

```
SELECT employee_id, last_name FROM Employees; -- sample pattern 38
```

39. Basic Q39: Show employee_id and last_name for employee #39 (example pattern).

```
SELECT employee_id, last_name FROM Employees; -- sample pattern 39
```

40. Basic Q40: Show employee_id and last_name for employee #40 (example pattern).

```
SELECT employee_id, last_name FROM Employees; -- sample pattern 40
```

41. Basic Q41: Show employee_id and last_name for employee #41 (example pattern).

```
SELECT employee_id, last_name FROM Employees; -- sample pattern 41
```

42. Basic Q42: Show employee_id and last_name for employee #42 (example pattern).

```
SELECT employee_id, last_name FROM Employees; -- sample pattern 42
```

43. Basic Q43: Show employee_id and last_name for employee #43 (example pattern).

```
SELECT employee_id, last_name FROM Employees; -- sample pattern 43
```

44. Basic Q44: Show employee_id and last_name for employee #44 (example pattern).

```
SELECT employee_id, last_name FROM Employees; -- sample pattern 44
```

45. Basic Q45: Show employee_id and last_name for employee #45 (example pattern).

```
SELECT employee_id, last_name FROM Employees; -- sample pattern 45
```

46. Basic Q46: Show employee_id and last_name for employee #46 (example pattern).

```
SELECT employee_id, last_name FROM Employees; -- sample pattern 46
```

47. Basic Q47: Show employee_id and last_name for employee #47 (example pattern).

```
SELECT employee_id, last_name FROM Employees; -- sample pattern 47
```

48. Basic Q48: Show employee_id and last_name for employee #48 (example pattern).

```
SELECT employee_id, last_name FROM Employees; -- sample pattern 48
```

49. Basic Q49: Show employee_id and last_name for employee #49 (example pattern).

```
SELECT employee_id, last_name FROM Employees; -- sample pattern 49
```

50. Basic Q50: Show employee_id and last_name for employee #50 (example pattern).

```
SELECT employee_id, last_name FROM Employees; -- sample pattern 50
```

Filtering with WHERE

1. Employees hired after 2020-01-01.

```
SELECT * FROM Employees WHERE hire_date > '2020-01-01';
```

2. Employees with salary > 50000.

```
SELECT * FROM Employees WHERE salary > 50000;
```

3. Employees whose name starts with 'A'.

```
SELECT * FROM Employees WHERE first_name LIKE 'A%';
```

4. Employees whose name ends with 'n'.

```
SELECT * FROM Employees WHERE first_name LIKE '%n';
```

5. Employees containing 'ar' in their name.

```
SELECT * FROM Employees WHERE first_name LIKE '%ar%';
```

6. Salary between 30000 and 60000.

```
SELECT * FROM Employees WHERE salary BETWEEN 30000 AND 60000;
```

7. Employees in department 20 or 30.

```
SELECT * FROM Employees WHERE department_id IN (20,30);
```

8. Employees without a manager.

```
SELECT * FROM Employees WHERE manager_id IS NULL;
```

9. Orders greater than 2000.

```
SELECT * FROM Orders WHERE amount > 2000;
```

10. Employees hired in 2022.

```
SELECT * FROM Employees WHERE YEAR(hire_date) = 2022;
```

11. Filter Q11: Show employees with salary > 31000.

```
SELECT * FROM Employees WHERE salary > 31000;
```

12. Filter Q12: Show employees with salary > 32000.

```
SELECT * FROM Employees WHERE salary > 32000;
```

13. Filter Q13: Show employees with salary > 33000.

```
SELECT * FROM Employees WHERE salary > 33000;
```

14. Filter Q14: Show employees with salary > 34000.

```
SELECT * FROM Employees WHERE salary > 34000;
```

15. Filter Q15: Show employees with salary > 35000.

```
SELECT * FROM Employees WHERE salary > 35000;
```

16. Filter Q16: Show employees with salary > 36000.

```
SELECT * FROM Employees WHERE salary > 36000;
```

17. Filter Q17: Show employees with salary > 37000.

```
SELECT * FROM Employees WHERE salary > 37000;
```

18. Filter Q18: Show employees with salary > 38000.

```
SELECT * FROM Employees WHERE salary > 38000;
```

19. Filter Q19: Show employees with salary > 39000.

```
SELECT * FROM Employees WHERE salary > 39000;
```

20. Filter Q20: Show employees with salary > 40000.

```
SELECT * FROM Employees WHERE salary > 40000;
```

21. Filter Q21: Show employees with salary > 41000.

```
SELECT * FROM Employees WHERE salary > 41000;
```

22. Filter Q22: Show employees with salary > 42000.

```
SELECT * FROM Employees WHERE salary > 42000;
```

23. Filter Q23: Show employees with salary > 43000.

```
SELECT * FROM Employees WHERE salary > 43000;
```

24. Filter Q24: Show employees with salary > 44000.

```
SELECT * FROM Employees WHERE salary > 44000;
```

25. Filter Q25: Show employees with salary > 45000.

```
SELECT * FROM Employees WHERE salary > 45000;
```

26. Filter Q26: Show employees with salary > 46000.

```
SELECT * FROM Employees WHERE salary > 46000;
```

27. Filter Q27: Show employees with salary > 47000.

```
SELECT * FROM Employees WHERE salary > 47000;
```

28. Filter Q28: Show employees with salary > 48000.

```
SELECT * FROM Employees WHERE salary > 48000;
```

29. Filter Q29: Show employees with salary > 49000.

```
SELECT * FROM Employees WHERE salary > 49000;
```

30. Filter Q30: Show employees with salary > 50000.

```
SELECT * FROM Employees WHERE salary > 50000;
```

31. Filter Q31: Show employees with salary > 51000.

```
SELECT * FROM Employees WHERE salary > 51000;
```

32. Filter Q32: Show employees with salary > 52000.

```
SELECT * FROM Employees WHERE salary > 52000;
```

33. Filter Q33: Show employees with salary > 53000.

```
SELECT * FROM Employees WHERE salary > 53000;
```

34. Filter Q34: Show employees with salary > 54000.

```
SELECT * FROM Employees WHERE salary > 54000;
```

35. Filter Q35: Show employees with salary > 55000.

```
SELECT * FROM Employees WHERE salary > 55000;
```

36. Filter Q36: Show employees with salary > 56000.

```
SELECT * FROM Employees WHERE salary > 56000;
```

37. Filter Q37: Show employees with salary > 57000.

```
SELECT * FROM Employees WHERE salary > 57000;
```

38. Filter Q38: Show employees with salary > 58000.

```
SELECT * FROM Employees WHERE salary > 58000;
```

39. Filter Q39: Show employees with salary > 59000.

```
SELECT * FROM Employees WHERE salary > 59000;
```

40. Filter Q40: Show employees with salary > 60000.

```
SELECT * FROM Employees WHERE salary > 60000;
```

41. Filter Q41: Show employees with salary > 61000.

```
SELECT * FROM Employees WHERE salary > 61000;
```

42. Filter Q42: Show employees with salary > 62000.

```
SELECT * FROM Employees WHERE salary > 62000;
```

43. Filter Q43: Show employees with salary > 63000.

```
SELECT * FROM Employees WHERE salary > 63000;
```

44. Filter Q44: Show employees with salary > 64000.

```
SELECT * FROM Employees WHERE salary > 64000;
```

45. Filter Q45: Show employees with salary > 65000.

```
SELECT * FROM Employees WHERE salary > 65000;
```

46. Filter Q46: Show employees with salary > 66000.

```
SELECT * FROM Employees WHERE salary > 66000;
```

47. Filter Q47: Show employees with salary > 67000.

```
SELECT * FROM Employees WHERE salary > 67000;
```

48. Filter Q48: Show employees with salary > 68000.

```
SELECT * FROM Employees WHERE salary > 68000;
```

49. Filter Q49: Show employees with salary > 69000.

```
SELECT * FROM Employees WHERE salary > 69000;
```

50. Filter Q50: Show employees with salary > 70000.

```
SELECT * FROM Employees WHERE salary > 70000;
```

Aggregates & GROUP BY

1. Count all employees.

```
SELECT COUNT(*) AS TotalEmployees FROM Employees;
```

2. Average salary of employees.

```
SELECT AVG(salary) AS AvgSalary FROM Employees;
```

3. Max salary in company.

```
SELECT MAX(salary) AS MaxSalary FROM Employees;
```

4. Min salary in company.

```
SELECT MIN(salary) AS MinSalary FROM Employees;
```

5. Sum of all salaries.

```
SELECT SUM(salary) AS TotalSalary FROM Employees;
```

6. Count employees per department.

```
SELECT department_id, COUNT(*) AS EmpCount FROM Employees GROUP BY department_id;
```

7. Average salary per department.

```
SELECT department_id, AVG(salary) AS AvgSalary FROM Employees GROUP BY department_id;
```

8. Departments with more than 5 employees.

```
SELECT department_id, COUNT(*) AS EmpCount FROM Employees GROUP BY department_id HAVING
COUNT(*) > 5;
```

9. Employees hired per year.

```
SELECT YEAR(hire_date) AS HireYear, COUNT(*) AS CountPerYear FROM Employees GROUP BY
YEAR(hire_date);
```

10. Total order amount per employee.

```
SELECT employee_id, SUM(amount) AS TotalOrders FROM Orders GROUP BY employee_id;
```

11. Aggregate Q11: Find average salary for department_id = 2.

```
SELECT AVG(salary) AS AvgSalary FROM Employees WHERE department_id = 2;
```

12. Aggregate Q12: Find average salary for department_id = 3.

```
SELECT AVG(salary) AS AvgSalary FROM Employees WHERE department_id = 3;
```

13. Aggregate Q13: Find average salary for department_id = 4.

```
SELECT AVG(salary) AS AvgSalary FROM Employees WHERE department_id = 4;
```

14. Aggregate Q14: Find average salary for department_id = 5.

```
SELECT AVG(salary) AS AvgSalary FROM Employees WHERE department_id = 5;
```

15. Aggregate Q15: Find average salary for department_id = 6.

```
SELECT AVG(salary) AS AvgSalary FROM Employees WHERE department_id = 6;
```

16. Aggregate Q16: Find average salary for department_id = 7.

```
SELECT AVG(salary) AS AvgSalary FROM Employees WHERE department_id = 7;
```

17. Aggregate Q17: Find average salary for department_id = 8.

```
SELECT AVG(salary) AS AvgSalary FROM Employees WHERE department_id = 8;
```

18. Aggregate Q18: Find average salary for department_id = 9.

```
SELECT AVG(salary) AS AvgSalary FROM Employees WHERE department_id = 9;
```

19. Aggregate Q19: Find average salary for department_id = 10.

```
SELECT AVG(salary) AS AvgSalary FROM Employees WHERE department_id = 10;
```

20. Aggregate Q20: Find average salary for department_id = 1.

```
SELECT AVG(salary) AS AvgSalary FROM Employees WHERE department_id = 1;
```

21. Aggregate Q21: Find average salary for department_id = 2.

```
SELECT AVG(salary) AS AvgSalary FROM Employees WHERE department_id = 2;
```

22. Aggregate Q22: Find average salary for department_id = 3.

```
SELECT AVG(salary) AS AvgSalary FROM Employees WHERE department_id = 3;
```

23. Aggregate Q23: Find average salary for department_id = 4.

```
SELECT AVG(salary) AS AvgSalary FROM Employees WHERE department_id = 4;
```

24. Aggregate Q24: Find average salary for department_id = 5.

```
SELECT AVG(salary) AS AvgSalary FROM Employees WHERE department_id = 5;
```

25. Aggregate Q25: Find average salary for department_id = 6.

```
SELECT AVG(salary) AS AvgSalary FROM Employees WHERE department_id = 6;
```

26. Aggregate Q26: Find average salary for department_id = 7.

```
SELECT AVG(salary) AS AvgSalary FROM Employees WHERE department_id = 7;
```

27. Aggregate Q27: Find average salary for department_id = 8.

```
SELECT AVG(salary) AS AvgSalary FROM Employees WHERE department_id = 8;
```

28. Aggregate Q28: Find average salary for department_id = 9.

```
SELECT AVG(salary) AS AvgSalary FROM Employees WHERE department_id = 9;
```

29. Aggregate Q29: Find average salary for department_id = 10.

```
SELECT AVG(salary) AS AvgSalary FROM Employees WHERE department_id = 10;
```

30. Aggregate Q30: Find average salary for department_id = 1.

```
SELECT AVG(salary) AS AvgSalary FROM Employees WHERE department_id = 1;
```

31. Aggregate Q31: Find average salary for department_id = 2.

```
SELECT AVG(salary) AS AvgSalary FROM Employees WHERE department_id = 2;
```

32. Aggregate Q32: Find average salary for department_id = 3.

```
SELECT AVG(salary) AS AvgSalary FROM Employees WHERE department_id = 3;
```

33. Aggregate Q33: Find average salary for department_id = 4.

```
SELECT AVG(salary) AS AvgSalary FROM Employees WHERE department_id = 4;
```

34. Aggregate Q34: Find average salary for department_id = 5.

```
SELECT AVG(salary) AS AvgSalary FROM Employees WHERE department_id = 5;
```

35. Aggregate Q35: Find average salary for department_id = 6.

```
SELECT AVG(salary) AS AvgSalary FROM Employees WHERE department_id = 6;
```

36. Aggregate Q36: Find average salary for department_id = 7.

```
SELECT AVG(salary) AS AvgSalary FROM Employees WHERE department_id = 7;
```

37. Aggregate Q37: Find average salary for department_id = 8.

```
SELECT AVG(salary) AS AvgSalary FROM Employees WHERE department_id = 8;
```

38. Aggregate Q38: Find average salary for department_id = 9.

```
SELECT AVG(salary) AS AvgSalary FROM Employees WHERE department_id = 9;
```

39. Aggregate Q39: Find average salary for department_id = 10.

```
SELECT AVG(salary) AS AvgSalary FROM Employees WHERE department_id = 10;
```

40. Aggregate Q40: Find average salary for department_id = 1.

```
SELECT AVG(salary) AS AvgSalary FROM Employees WHERE department_id = 1;
```

41. Aggregate Q41: Find average salary for department_id = 2.

```
SELECT AVG(salary) AS AvgSalary FROM Employees WHERE department_id = 2;
```

42. Aggregate Q42: Find average salary for department_id = 3.

```
SELECT AVG(salary) AS AvgSalary FROM Employees WHERE department_id = 3;
```

43. Aggregate Q43: Find average salary for department_id = 4.

```
SELECT AVG(salary) AS AvgSalary FROM Employees WHERE department_id = 4;
```

44. Aggregate Q44: Find average salary for department_id = 5.

```
SELECT AVG(salary) AS AvgSalary FROM Employees WHERE department_id = 5;
```

45. Aggregate Q45: Find average salary for department_id = 6.

```
SELECT AVG(salary) AS AvgSalary FROM Employees WHERE department_id = 6;
```

46. Aggregate Q46: Find average salary for department_id = 7.

```
SELECT AVG(salary) AS AvgSalary FROM Employees WHERE department_id = 7;
```

47. Aggregate Q47: Find average salary for department_id = 8.

```
SELECT AVG(salary) AS AvgSalary FROM Employees WHERE department_id = 8;
```

48. Aggregate Q48: Find average salary for department_id = 9.

```
SELECT AVG(salary) AS AvgSalary FROM Employees WHERE department_id = 9;
```

49. Aggregate Q49: Find average salary for department_id = 10.

```
SELECT AVG(salary) AS AvgSalary FROM Employees WHERE department_id = 10;
```

50. Aggregate Q50: Find average salary for department_id = 1.

```
SELECT AVG(salary) AS AvgSalary FROM Employees WHERE department_id = 1;
```

Joins

1. Employees with their department name.

```
SELECT e.*, d.department_name FROM Employees e JOIN Departments d ON e.department_id = d.department_id;
```

2. Employees with their manager's name (self join).

```
SELECT e.employee_id, e.first_name AS Employee, m.first_name AS Manager FROM Employees e LEFT JOIN Employees m ON e.manager_id = m.employee_id;
```

3. Employees with their orders.

```
SELECT e.first_name, o.order_id, o.amount FROM Employees e JOIN Orders o ON e.employee_id = o.employee_id;
```

4. Orders with employee and department.

```
SELECT o.order_id, e.first_name, d.department_name FROM Orders o JOIN Employees e ON o.employee_id = e.employee_id JOIN Departments d ON e.department_id = d.department_id;
```

5. Departments with no employees (LEFT JOIN).

```
SELECT d.department_name FROM Departments d LEFT JOIN Employees e ON d.department_id = e.department_id WHERE e.employee_id IS NULL;
```

6. Employees without orders (LEFT JOIN + NULL).

```
SELECT e.* FROM Employees e LEFT JOIN Orders o ON e.employee_id = o.employee_id WHERE o.order_id IS NULL;
```

7. Join 3 tables sample.

```
SELECT e.first_name, d.department_name, SUM(o.amount) AS TotalSales FROM Employees e JOIN Departments d ON e.department_id = d.department_id JOIN Orders o ON e.employee_id = o.employee_id GROUP BY e.first_name, d.department_name;
```

8. Self-join to show employee hierarchy.

```
SELECT e.employee_id, e.first_name, m.employee_id AS manager_id, m.first_name AS manager_name FROM Employees e LEFT JOIN Employees m ON e.manager_id = m.employee_id;
```

9. Orders per department.

```
SELECT d.department_name, SUM(o.amount) AS DeptSales FROM Departments d JOIN Employees e ON d.department_id = e.department_id JOIN Orders o ON e.employee_id = o.employee_id GROUP BY d.department_name;
```

10. Employees and their manager chain (two levels).

```
SELECT e.first_name AS Employee, m.first_name AS Manager, mm.first_name AS ManagerManager FROM Employees e LEFT JOIN Employees m ON e.manager_id = m.employee_id LEFT JOIN Employees mm ON m.manager_id = mm.employee_id;
```

11. Join Q11: Show orders and employee names for orders above 200.

```
SELECT o.order_id, o.amount, e.first_name FROM Orders o JOIN Employees e ON o.employee_id = e.employee_id WHERE o.amount > 200;
```

12. Join Q12: Show orders and employee names for orders above 300.

```
SELECT o.order_id, o.amount, e.first_name FROM Orders o JOIN Employees e ON o.employee_id = e.employee_id WHERE o.amount > 300;
```

13. Join Q13: Show orders and employee names for orders above 400.

```
SELECT o.order_id, o.amount, e.first_name FROM Orders o JOIN Employees e ON
o.employee_id = e.employee_id WHERE o.amount > 400;
```

14. Join Q14: Show orders and employee names for orders above 500.

```
SELECT o.order_id, o.amount, e.first_name FROM Orders o JOIN Employees e ON
o.employee_id = e.employee_id WHERE o.amount > 500;
```

15. Join Q15: Show orders and employee names for orders above 100.

```
SELECT o.order_id, o.amount, e.first_name FROM Orders o JOIN Employees e ON
o.employee_id = e.employee_id WHERE o.amount > 100;
```

16. Join Q16: Show orders and employee names for orders above 200.

```
SELECT o.order_id, o.amount, e.first_name FROM Orders o JOIN Employees e ON
o.employee_id = e.employee_id WHERE o.amount > 200;
```

17. Join Q17: Show orders and employee names for orders above 300.

```
SELECT o.order_id, o.amount, e.first_name FROM Orders o JOIN Employees e ON
o.employee_id = e.employee_id WHERE o.amount > 300;
```

18. Join Q18: Show orders and employee names for orders above 400.

```
SELECT o.order_id, o.amount, e.first_name FROM Orders o JOIN Employees e ON
o.employee_id = e.employee_id WHERE o.amount > 400;
```

19. Join Q19: Show orders and employee names for orders above 500.

```
SELECT o.order_id, o.amount, e.first_name FROM Orders o JOIN Employees e ON
o.employee_id = e.employee_id WHERE o.amount > 500;
```

20. Join Q20: Show orders and employee names for orders above 100.

```
SELECT o.order_id, o.amount, e.first_name FROM Orders o JOIN Employees e ON
o.employee_id = e.employee_id WHERE o.amount > 100;
```

21. Join Q21: Show orders and employee names for orders above 200.

```
SELECT o.order_id, o.amount, e.first_name FROM Orders o JOIN Employees e ON
o.employee_id = e.employee_id WHERE o.amount > 200;
```

22. Join Q22: Show orders and employee names for orders above 300.

```
SELECT o.order_id, o.amount, e.first_name FROM Orders o JOIN Employees e ON
o.employee_id = e.employee_id WHERE o.amount > 300;
```

23. Join Q23: Show orders and employee names for orders above 400.

```
SELECT o.order_id, o.amount, e.first_name FROM Orders o JOIN Employees e ON
o.employee_id = e.employee_id WHERE o.amount > 400;
```

24. Join Q24: Show orders and employee names for orders above 500.

```
SELECT o.order_id, o.amount, e.first_name FROM Orders o JOIN Employees e ON
o.employee_id = e.employee_id WHERE o.amount > 500;
```

25. Join Q25: Show orders and employee names for orders above 100.

```
SELECT o.order_id, o.amount, e.first_name FROM Orders o JOIN Employees e ON
o.employee_id = e.employee_id WHERE o.amount > 100;
```

26. Join Q26: Show orders and employee names for orders above 200.

```
SELECT o.order_id, o.amount, e.first_name FROM Orders o JOIN Employees e ON
o.employee_id = e.employee_id WHERE o.amount > 200;
```

27. Join Q27: Show orders and employee names for orders above 300.

```
SELECT o.order_id, o.amount, e.first_name FROM Orders o JOIN Employees e ON
o.employee_id = e.employee_id WHERE o.amount > 300;
```

28. Join Q28: Show orders and employee names for orders above 400.

```
SELECT o.order_id, o.amount, e.first_name FROM Orders o JOIN Employees e ON
o.employee_id = e.employee_id WHERE o.amount > 400;
```

29. Join Q29: Show orders and employee names for orders above 500.

```
SELECT o.order_id, o.amount, e.first_name FROM Orders o JOIN Employees e ON
o.employee_id = e.employee_id WHERE o.amount > 500;
```

30. Join Q30: Show orders and employee names for orders above 100.

```
SELECT o.order_id, o.amount, e.first_name FROM Orders o JOIN Employees e ON
o.employee_id = e.employee_id WHERE o.amount > 100;
```

31. Join Q31: Show orders and employee names for orders above 200.

```
SELECT o.order_id, o.amount, e.first_name FROM Orders o JOIN Employees e ON
o.employee_id = e.employee_id WHERE o.amount > 200;
```

32. Join Q32: Show orders and employee names for orders above 300.

```
SELECT o.order_id, o.amount, e.first_name FROM Orders o JOIN Employees e ON
o.employee_id = e.employee_id WHERE o.amount > 300;
```

33. Join Q33: Show orders and employee names for orders above 400.

```
SELECT o.order_id, o.amount, e.first_name FROM Orders o JOIN Employees e ON
o.employee_id = e.employee_id WHERE o.amount > 400;
```

34. Join Q34: Show orders and employee names for orders above 500.

```
SELECT o.order_id, o.amount, e.first_name FROM Orders o JOIN Employees e ON
o.employee_id = e.employee_id WHERE o.amount > 500;
```

35. Join Q35: Show orders and employee names for orders above 100.

```
SELECT o.order_id, o.amount, e.first_name FROM Orders o JOIN Employees e ON
o.employee_id = e.employee_id WHERE o.amount > 100;
```

36. Join Q36: Show orders and employee names for orders above 200.

```
SELECT o.order_id, o.amount, e.first_name FROM Orders o JOIN Employees e ON
o.employee_id = e.employee_id WHERE o.amount > 200;
```

37. Join Q37: Show orders and employee names for orders above 300.

```
SELECT o.order_id, o.amount, e.first_name FROM Orders o JOIN Employees e ON
o.employee_id = e.employee_id WHERE o.amount > 300;
```

38. Join Q38: Show orders and employee names for orders above 400.

```
SELECT o.order_id, o.amount, e.first_name FROM Orders o JOIN Employees e ON
o.employee_id = e.employee_id WHERE o.amount > 400;
```

39. Join Q39: Show orders and employee names for orders above 500.

```
SELECT o.order_id, o.amount, e.first_name FROM Orders o JOIN Employees e ON
o.employee_id = e.employee_id WHERE o.amount > 500;
```

40. Join Q40: Show orders and employee names for orders above 100.

```
SELECT o.order_id, o.amount, e.first_name FROM Orders o JOIN Employees e ON
o.employee_id = e.employee_id WHERE o.amount > 100;
```

41. Join Q41: Show orders and employee names for orders above 200.

```
SELECT o.order_id, o.amount, e.first_name FROM Orders o JOIN Employees e ON
o.employee_id = e.employee_id WHERE o.amount > 200;
```

42. Join Q42: Show orders and employee names for orders above 300.

```
SELECT o.order_id, o.amount, e.first_name FROM Orders o JOIN Employees e ON
o.employee_id = e.employee_id WHERE o.amount > 300;
```

43. Join Q43: Show orders and employee names for orders above 400.

```
SELECT o.order_id, o.amount, e.first_name FROM Orders o JOIN Employees e ON
o.employee_id = e.employee_id WHERE o.amount > 400;
```

44. Join Q44: Show orders and employee names for orders above 500.

```
SELECT o.order_id, o.amount, e.first_name FROM Orders o JOIN Employees e ON
o.employee_id = e.employee_id WHERE o.amount > 500;
```

45. Join Q45: Show orders and employee names for orders above 100.

```
SELECT o.order_id, o.amount, e.first_name FROM Orders o JOIN Employees e ON
o.employee_id = e.employee_id WHERE o.amount > 100;
```

46. Join Q46: Show orders and employee names for orders above 200.

```
SELECT o.order_id, o.amount, e.first_name FROM Orders o JOIN Employees e ON
o.employee_id = e.employee_id WHERE o.amount > 200;
```

47. Join Q47: Show orders and employee names for orders above 300.

```
SELECT o.order_id, o.amount, e.first_name FROM Orders o JOIN Employees e ON
o.employee_id = e.employee_id WHERE o.amount > 300;
```

48. Join Q48: Show orders and employee names for orders above 400.

```
SELECT o.order_id, o.amount, e.first_name FROM Orders o JOIN Employees e ON
o.employee_id = e.employee_id WHERE o.amount > 400;
```

49. Join Q49: Show orders and employee names for orders above 500.

```
SELECT o.order_id, o.amount, e.first_name FROM Orders o JOIN Employees e ON
o.employee_id = e.employee_id WHERE o.amount > 500;
```

50. Join Q50: Show orders and employee names for orders above 100.

```
SELECT o.order_id, o.amount, e.first_name FROM Orders o JOIN Employees e ON
o.employee_id = e.employee_id WHERE o.amount > 100;
```

Subqueries

1. Employees earning more than company avg.

```
SELECT * FROM Employees WHERE salary > (SELECT AVG(salary) FROM Employees);
```

2. Employees earning more than ALL in dept 30.

```
SELECT * FROM Employees WHERE salary > ALL (SELECT salary FROM Employees WHERE
department_id = 30);
```

3. Employees earning more than ANY in dept 40.

```
SELECT * FROM Employees WHERE salary > ANY (SELECT salary FROM Employees WHERE
department_id = 40);
```

4. Employees in same dept as 'John'.

```
SELECT * FROM Employees WHERE department_id = (SELECT department_id FROM Employees
WHERE first_name = 'John');
```

5. Employees whose salary equals max salary.

```
SELECT * FROM Employees WHERE salary = (SELECT MAX(salary) FROM Employees);
```

6. Employees earning > dept avg (correlated).

```
SELECT e.* FROM Employees e WHERE e.salary > (SELECT AVG(salary) FROM Employees WHERE
department_id = e.department_id);
```

7. Orders above company avg order amount.

```
SELECT * FROM Orders WHERE amount > (SELECT AVG(amount) FROM Orders);
```

8. Employees with salary greater than their manager.

```
SELECT e.* FROM Employees e JOIN Employees m ON e.manager_id = m.employee_id WHERE
e.salary > m.salary;
```

9. Departments with no employees using NOT EXISTS.

```
SELECT * FROM Departments d WHERE NOT EXISTS (SELECT 1 FROM Employees e WHERE
e.department_id = d.department_id);
```

10. Employees with second highest salary.

```
SELECT * FROM Employees WHERE salary = (SELECT MAX(salary) FROM Employees WHERE salary
< (SELECT MAX(salary) FROM Employees));
```

11. Subquery Q11: Employees earning more than the average of department 2.

```
SELECT * FROM Employees WHERE salary > (SELECT AVG(salary) FROM Employees WHERE
department_id = 2);
```

12. Subquery Q12: Employees earning more than the average of department 3.

```
SELECT * FROM Employees WHERE salary > (SELECT AVG(salary) FROM Employees WHERE
department_id = 3);
```

13. Subquery Q13: Employees earning more than the average of department 4.

```
SELECT * FROM Employees WHERE salary > (SELECT AVG(salary) FROM Employees WHERE
department_id = 4);
```

14. Subquery Q14: Employees earning more than the average of department 5.

```
SELECT * FROM Employees WHERE salary > (SELECT AVG(salary) FROM Employees WHERE
department_id = 5);
```

15. Subquery Q15: Employees earning more than the average of department 6.

```
SELECT * FROM Employees WHERE salary > (SELECT AVG(salary) FROM Employees WHERE
department_id = 6);
```

16. Subquery Q16: Employees earning more than the average of department 7.

```
SELECT * FROM Employees WHERE salary > (SELECT AVG(salary) FROM Employees WHERE
department_id = 7);
```

17. Subquery Q17: Employees earning more than the average of department 8.

```
SELECT * FROM Employees WHERE salary > (SELECT AVG(salary) FROM Employees WHERE
department_id = 8);
```

18. Subquery Q18: Employees earning more than the average of department 9.

```
SELECT * FROM Employees WHERE salary > (SELECT AVG(salary) FROM Employees WHERE
department_id = 9);
```

19. Subquery Q19: Employees earning more than the average of department 10.

```
SELECT * FROM Employees WHERE salary > (SELECT AVG(salary) FROM Employees WHERE
department_id = 10);
```

20. Subquery Q20: Employees earning more than the average of department 1.

```
SELECT * FROM Employees WHERE salary > (SELECT AVG(salary) FROM Employees WHERE
department_id = 1);
```

21. Subquery Q21: Employees earning more than the average of department 2.

```
SELECT * FROM Employees WHERE salary > (SELECT AVG(salary) FROM Employees WHERE
department_id = 2);
```

22. Subquery Q22: Employees earning more than the average of department 3.

```
SELECT * FROM Employees WHERE salary > (SELECT AVG(salary) FROM Employees WHERE
department_id = 3);
```

23. Subquery Q23: Employees earning more than the average of department 4.

```
SELECT * FROM Employees WHERE salary > (SELECT AVG(salary) FROM Employees WHERE
department_id = 4);
```

24. Subquery Q24: Employees earning more than the average of department 5.

```
SELECT * FROM Employees WHERE salary > (SELECT AVG(salary) FROM Employees WHERE
department_id = 5);
```

25. Subquery Q25: Employees earning more than the average of department 6.

```
SELECT * FROM Employees WHERE salary > (SELECT AVG(salary) FROM Employees WHERE
department_id = 6);
```

26. Subquery Q26: Employees earning more than the average of department 7.

```
SELECT * FROM Employees WHERE salary > (SELECT AVG(salary) FROM Employees WHERE
department_id = 7);
```

27. Subquery Q27: Employees earning more than the average of department 8.

```
SELECT * FROM Employees WHERE salary > (SELECT AVG(salary) FROM Employees WHERE
department_id = 8);
```

28. Subquery Q28: Employees earning more than the average of department 9.

```
SELECT * FROM Employees WHERE salary > (SELECT AVG(salary) FROM Employees WHERE
department_id = 9);
```

29. Subquery Q29: Employees earning more than the average of department 10.

```
SELECT * FROM Employees WHERE salary > (SELECT AVG(salary) FROM Employees WHERE
department_id = 10);
```

30. Subquery Q30: Employees earning more than the average of department 1.

```
SELECT * FROM Employees WHERE salary > (SELECT AVG(salary) FROM Employees WHERE
department_id = 1);
```

31. Subquery Q31: Employees earning more than the average of department 2.

```
SELECT * FROM Employees WHERE salary > (SELECT AVG(salary) FROM Employees WHERE
department_id = 2);
```

32. Subquery Q32: Employees earning more than the average of department 3.

```
SELECT * FROM Employees WHERE salary > (SELECT AVG(salary) FROM Employees WHERE
department_id = 3);
```

33. Subquery Q33: Employees earning more than the average of department 4.

```
SELECT * FROM Employees WHERE salary > (SELECT AVG(salary) FROM Employees WHERE
department_id = 4);
```

34. Subquery Q34: Employees earning more than the average of department 5.

```
SELECT * FROM Employees WHERE salary > (SELECT AVG(salary) FROM Employees WHERE
department_id = 5);
```

35. Subquery Q35: Employees earning more than the average of department 6.

```
SELECT * FROM Employees WHERE salary > (SELECT AVG(salary) FROM Employees WHERE
department_id = 6);
```

36. Subquery Q36: Employees earning more than the average of department 7.

```
SELECT * FROM Employees WHERE salary > (SELECT AVG(salary) FROM Employees WHERE
department_id = 7);
```

37. Subquery Q37: Employees earning more than the average of department 8.

```
SELECT * FROM Employees WHERE salary > (SELECT AVG(salary) FROM Employees WHERE
department_id = 8);
```

38. Subquery Q38: Employees earning more than the average of department 9.

```
SELECT * FROM Employees WHERE salary > (SELECT AVG(salary) FROM Employees WHERE
department_id = 9);
```

39. Subquery Q39: Employees earning more than the average of department 10.

```
SELECT * FROM Employees WHERE salary > (SELECT AVG(salary) FROM Employees WHERE
department_id = 10);
```

40. Subquery Q40: Employees earning more than the average of department 1.

```
SELECT * FROM Employees WHERE salary > (SELECT AVG(salary) FROM Employees WHERE
department_id = 1);
```

41. Subquery Q41: Employees earning more than the average of department 2.

```
SELECT * FROM Employees WHERE salary > (SELECT AVG(salary) FROM Employees WHERE
department_id = 2);
```

42. Subquery Q42: Employees earning more than the average of department 3.

```
SELECT * FROM Employees WHERE salary > (SELECT AVG(salary) FROM Employees WHERE
department_id = 3);
```

43. Subquery Q43: Employees earning more than the average of department 4.

```
SELECT * FROM Employees WHERE salary > (SELECT AVG(salary) FROM Employees WHERE
department_id = 4);
```

44. Subquery Q44: Employees earning more than the average of department 5.

```
SELECT * FROM Employees WHERE salary > (SELECT AVG(salary) FROM Employees WHERE
department_id = 5);
```

45. Subquery Q45: Employees earning more than the average of department 6.

```
SELECT * FROM Employees WHERE salary > (SELECT AVG(salary) FROM Employees WHERE
department_id = 6);
```

46. Subquery Q46: Employees earning more than the average of department 7.

```
SELECT * FROM Employees WHERE salary > (SELECT AVG(salary) FROM Employees WHERE
department_id = 7);
```

47. Subquery Q47: Employees earning more than the average of department 8.

```
SELECT * FROM Employees WHERE salary > (SELECT AVG(salary) FROM Employees WHERE
department_id = 8);
```

48. Subquery Q48: Employees earning more than the average of department 9.

```
SELECT * FROM Employees WHERE salary > (SELECT AVG(salary) FROM Employees WHERE
department_id = 9);
```

49. Subquery Q49: Employees earning more than the average of department 10.

```
SELECT * FROM Employees WHERE salary > (SELECT AVG(salary) FROM Employees WHERE
department_id = 10);
```

50. Subquery Q50: Employees earning more than the average of department 1.

```
SELECT * FROM Employees WHERE salary > (SELECT AVG(salary) FROM Employees WHERE
department_id = 1);
```

String & Date Functions

1. Employee names in UPPER.

```
SELECT UPPER(first_name + ' ' + last_name) AS FullName FROM Employees;
```

2. Employee names in LOWER.

```
SELECT LOWER(first_name + ' ' + last_name) AS FullName FROM Employees;
```

3. Length of first name.

```
SELECT first_name, LEN(first_name) AS NameLength FROM Employees;
```

4. First 3 letters of last name.

```
SELECT SUBSTRING(last_name,1,3) AS First3 FROM Employees;
```

5. Replace 'a' with '@' in names.

```
SELECT REPLACE(first_name, 'a', '@') FROM Employees;
```

6. Reverse first name.

```
SELECT REVERSE(first_name) FROM Employees;
```

7. Employees hired in 2022.

```
SELECT * FROM Employees WHERE YEAR(hire_date) = 2022;
```

8. Extract month from hire_date.

```
SELECT first_name, MONTH(hire_date) AS HireMonth FROM Employees;
```

9. Add 30 days to hire_date.

```
SELECT first_name, DATEADD(day,30,hire_date) AS Plus30 FROM Employees;
```

10. Years of service (integer).

```
SELECT first_name, DATEDIFF(year, hire_date, GETDATE()) AS YearsService FROM Employees;
```

11. Func Q11: Show first 2 letters of first_name.

```
SELECT SUBSTRING(first_name,1,2) FROM Employees;
```

12. Func Q12: Show first 2 letters of first_name.

```
SELECT SUBSTRING(first_name,1,2) FROM Employees;
```

13. Func Q13: Show first 2 letters of first_name.

```
SELECT SUBSTRING(first_name,1,2) FROM Employees;
```

14. Func Q14: Show first 2 letters of first_name.

```
SELECT SUBSTRING(first_name,1,2) FROM Employees;
```

15. Func Q15: Show first 2 letters of first_name.

```
SELECT SUBSTRING(first_name,1,2) FROM Employees;
```

16. Func Q16: Show first 2 letters of first_name.

```
SELECT SUBSTRING(first_name,1,2) FROM Employees;
```

17. Func Q17: Show first 2 letters of first_name.

```
SELECT SUBSTRING(first_name,1,2) FROM Employees;
```

18. Func Q18: Show first 2 letters of first_name.

```
SELECT SUBSTRING(first_name,1,2) FROM Employees;
```

19. Func Q19: Show first 2 letters of first_name.

```
SELECT SUBSTRING(first_name,1,2) FROM Employees;
```

20. Func Q20: Show first 2 letters of first_name.

```
SELECT SUBSTRING(first_name,1,2) FROM Employees;
```

21. Func Q21: Show first 2 letters of first_name.

```
SELECT SUBSTRING(first_name,1,2) FROM Employees;
```

22. Func Q22: Show first 2 letters of first_name.

```
SELECT SUBSTRING(first_name,1,2) FROM Employees;
```

23. Func Q23: Show first 2 letters of first_name.

```
SELECT SUBSTRING(first_name,1,2) FROM Employees;
```

24. Func Q24: Show first 2 letters of first_name.

```
SELECT SUBSTRING(first_name,1,2) FROM Employees;
```

25. Func Q25: Show first 2 letters of first_name.

```
SELECT SUBSTRING(first_name,1,2) FROM Employees;
```

26. Func Q26: Show first 2 letters of first_name.

```
SELECT SUBSTRING(first_name,1,2) FROM Employees;
```

27. Func Q27: Show first 2 letters of first_name.

```
SELECT SUBSTRING(first_name,1,2) FROM Employees;
```

28. Func Q28: Show first 2 letters of first_name.

```
SELECT SUBSTRING(first_name,1,2) FROM Employees;
```

29. Func Q29: Show first 2 letters of first_name.

```
SELECT SUBSTRING(first_name,1,2) FROM Employees;
```

30. Func Q30: Show first 2 letters of first_name.

```
SELECT SUBSTRING(first_name,1,2) FROM Employees;
```

31. Func Q31: Show first 2 letters of first_name.

```
SELECT SUBSTRING(first_name,1,2) FROM Employees;
```

32. Func Q32: Show first 2 letters of first_name.

```
SELECT SUBSTRING(first_name,1,2) FROM Employees;
```

33. Func Q33: Show first 2 letters of first_name.

```
SELECT SUBSTRING(first_name,1,2) FROM Employees;
```

34. Func Q34: Show first 2 letters of first_name.

```
SELECT SUBSTRING(first_name,1,2) FROM Employees;
```

35. Func Q35: Show first 2 letters of first_name.

```
SELECT SUBSTRING(first_name,1,2) FROM Employees;
```

36. Func Q36: Show first 2 letters of first_name.

```
SELECT SUBSTRING(first_name,1,2) FROM Employees;
```

37. Func Q37: Show first 2 letters of first_name.

```
SELECT SUBSTRING(first_name,1,2) FROM Employees;
```

38. Func Q38: Show first 2 letters of first_name.

```
SELECT SUBSTRING(first_name,1,2) FROM Employees;
```

39. Func Q39: Show first 2 letters of first_name.

```
SELECT SUBSTRING(first_name,1,2) FROM Employees;
```

40. Func Q40: Show first 2 letters of first_name.

```
SELECT SUBSTRING(first_name,1,2) FROM Employees;
```

41. Func Q41: Show first 2 letters of first_name.

```
SELECT SUBSTRING(first_name,1,2) FROM Employees;
```

42. Func Q42: Show first 2 letters of first_name.

```
SELECT SUBSTRING(first_name,1,2) FROM Employees;
```

43. Func Q43: Show first 2 letters of first_name.

```
SELECT SUBSTRING(first_name,1,2) FROM Employees;
```

44. Func Q44: Show first 2 letters of first_name.

```
SELECT SUBSTRING(first_name,1,2) FROM Employees;
```

45. Func Q45: Show first 2 letters of first_name.

```
SELECT SUBSTRING(first_name,1,2) FROM Employees;
```

46. Func Q46: Show first 2 letters of first_name.

```
SELECT SUBSTRING(first_name,1,2) FROM Employees;
```

47. Func Q47: Show first 2 letters of first_name.

```
SELECT SUBSTRING(first_name,1,2) FROM Employees;
```

48. Func Q48: Show first 2 letters of first_name.

```
SELECT SUBSTRING(first_name,1,2) FROM Employees;
```

49. Func Q49: Show first 2 letters of first_name.

```
SELECT SUBSTRING(first_name,1,2) FROM Employees;
```

50. Func Q50: Show first 2 letters of first_name.

```
SELECT SUBSTRING(first_name,1,2) FROM Employees;
```

Advanced

1. Second highest salary.

```
SELECT MAX(salary) AS SecondHighest FROM Employees WHERE salary < (SELECT MAX(salary) FROM Employees);
```

2. Third highest salary.

```
WITH Ranked AS (SELECT salary, ROW_NUMBER() OVER (ORDER BY salary DESC) rn FROM Employees) SELECT salary FROM Ranked WHERE rn = 3;
```

3. Nth highest salary (example N=4).

```
DECLARE @N INT = 4; SELECT MIN(salary) AS NthHighest FROM (SELECT DISTINCT TOP (@N) salary FROM Employees ORDER BY salary DESC) t;
```

4. Top 3 salaries per department.

```
SELECT * FROM (SELECT e.*, ROW_NUMBER() OVER (PARTITION BY department_id ORDER BY salary DESC) rn FROM Employees e) t WHERE rn <= 3;
```

5. Running total of salaries.

```
SELECT e.*, SUM(salary) OVER (ORDER BY salary DESC ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS RunningTotal FROM Employees e;
```

6. Rank employees by salary.

```
SELECT first_name, salary, RANK() OVER (ORDER BY salary DESC) AS SalaryRank FROM Employees;
```

7. Dense rank of salaries.

```
SELECT first_name, salary, DENSE_RANK() OVER (ORDER BY salary DESC) AS DenseRank FROM Employees;
```

8. Employees with duplicate salaries.

```
SELECT salary, COUNT(*) FROM Employees GROUP BY salary HAVING COUNT(*) > 1;
```

9. Recursive CTE for manager hierarchy.

```
WITH CTE AS (SELECT employee_id, first_name, manager_id FROM Employees WHERE manager_id IS NULL UNION ALL SELECT e.employee_id, e.first_name, e.manager_id FROM Employees e JOIN CTE c ON e.manager_id = c.employee_id) SELECT * FROM CTE;
```

10. Pivot example: employees by hire year (counts).

```
SELECT * FROM (SELECT first_name, YEAR(hire_date) AS yr FROM Employees) src PIVOT (COUNT(first_name) FOR yr IN ([2020],[2021],[2022],[2023])) p;
```

11. Advanced Q11: Use ROW_NUMBER to get top 2 per department (pattern).

```
SELECT * FROM (SELECT e.*, ROW_NUMBER() OVER (PARTITION BY department_id ORDER BY salary DESC) rn FROM Employees e) t WHERE rn <= 2;
```

12. Advanced Q12: Use ROW_NUMBER to get top 2 per department (pattern).

```
SELECT * FROM (SELECT e.*, ROW_NUMBER() OVER (PARTITION BY department_id ORDER BY salary DESC) rn FROM Employees e) t WHERE rn <= 2;
```

13. Advanced Q13: Use ROW_NUMBER to get top 2 per department (pattern).

```
SELECT * FROM (SELECT e.*, ROW_NUMBER() OVER (PARTITION BY department_id ORDER BY
salary DESC) rn FROM Employees e) t WHERE rn <= 2;
```

14. Advanced Q14: Use ROW_NUMBER to get top 2 per department (pattern).

```
SELECT * FROM (SELECT e.*, ROW_NUMBER() OVER (PARTITION BY department_id ORDER BY
salary DESC) rn FROM Employees e) t WHERE rn <= 2;
```

15. Advanced Q15: Use ROW_NUMBER to get top 2 per department (pattern).

```
SELECT * FROM (SELECT e.*, ROW_NUMBER() OVER (PARTITION BY department_id ORDER BY
salary DESC) rn FROM Employees e) t WHERE rn <= 2;
```

16. Advanced Q16: Use ROW_NUMBER to get top 2 per department (pattern).

```
SELECT * FROM (SELECT e.*, ROW_NUMBER() OVER (PARTITION BY department_id ORDER BY
salary DESC) rn FROM Employees e) t WHERE rn <= 2;
```

17. Advanced Q17: Use ROW_NUMBER to get top 2 per department (pattern).

```
SELECT * FROM (SELECT e.*, ROW_NUMBER() OVER (PARTITION BY department_id ORDER BY
salary DESC) rn FROM Employees e) t WHERE rn <= 2;
```

18. Advanced Q18: Use ROW_NUMBER to get top 2 per department (pattern).

```
SELECT * FROM (SELECT e.*, ROW_NUMBER() OVER (PARTITION BY department_id ORDER BY
salary DESC) rn FROM Employees e) t WHERE rn <= 2;
```

19. Advanced Q19: Use ROW_NUMBER to get top 2 per department (pattern).

```
SELECT * FROM (SELECT e.*, ROW_NUMBER() OVER (PARTITION BY department_id ORDER BY
salary DESC) rn FROM Employees e) t WHERE rn <= 2;
```

20. Advanced Q20: Use ROW_NUMBER to get top 2 per department (pattern).

```
SELECT * FROM (SELECT e.*, ROW_NUMBER() OVER (PARTITION BY department_id ORDER BY
salary DESC) rn FROM Employees e) t WHERE rn <= 2;
```

21. Advanced Q21: Use ROW_NUMBER to get top 2 per department (pattern).

```
SELECT * FROM (SELECT e.*, ROW_NUMBER() OVER (PARTITION BY department_id ORDER BY
salary DESC) rn FROM Employees e) t WHERE rn <= 2;
```

22. Advanced Q22: Use ROW_NUMBER to get top 2 per department (pattern).

```
SELECT * FROM (SELECT e.*, ROW_NUMBER() OVER (PARTITION BY department_id ORDER BY
salary DESC) rn FROM Employees e) t WHERE rn <= 2;
```

23. Advanced Q23: Use ROW_NUMBER to get top 2 per department (pattern).

```
SELECT * FROM (SELECT e.*, ROW_NUMBER() OVER (PARTITION BY department_id ORDER BY
salary DESC) rn FROM Employees e) t WHERE rn <= 2;
```

24. Advanced Q24: Use ROW_NUMBER to get top 2 per department (pattern).

```
SELECT * FROM (SELECT e.*, ROW_NUMBER() OVER (PARTITION BY department_id ORDER BY
salary DESC) rn FROM Employees e) t WHERE rn <= 2;
```

25. Advanced Q25: Use ROW_NUMBER to get top 2 per department (pattern).

```
SELECT * FROM (SELECT e.*, ROW_NUMBER() OVER (PARTITION BY department_id ORDER BY
salary DESC) rn FROM Employees e) t WHERE rn <= 2;
```

26. Advanced Q26: Use ROW_NUMBER to get top 2 per department (pattern).

```
SELECT * FROM (SELECT e.*, ROW_NUMBER() OVER (PARTITION BY department_id ORDER BY
salary DESC) rn FROM Employees e) t WHERE rn <= 2;
```

27. Advanced Q27: Use ROW_NUMBER to get top 2 per department (pattern).

```
SELECT * FROM (SELECT e.*, ROW_NUMBER() OVER (PARTITION BY department_id ORDER BY
salary DESC) rn FROM Employees e) t WHERE rn <= 2;
```

28. Advanced Q28: Use ROW_NUMBER to get top 2 per department (pattern).

```
SELECT * FROM (SELECT e.*, ROW_NUMBER() OVER (PARTITION BY department_id ORDER BY
salary DESC) rn FROM Employees e) t WHERE rn <= 2;
```

29. Advanced Q29: Use ROW_NUMBER to get top 2 per department (pattern).

```
SELECT * FROM (SELECT e.*, ROW_NUMBER() OVER (PARTITION BY department_id ORDER BY
salary DESC) rn FROM Employees e) t WHERE rn <= 2;
```

30. Advanced Q30: Use ROW_NUMBER to get top 2 per department (pattern).

```
SELECT * FROM (SELECT e.*, ROW_NUMBER() OVER (PARTITION BY department_id ORDER BY
salary DESC) rn FROM Employees e) t WHERE rn <= 2;
```

31. Advanced Q31: Use ROW_NUMBER to get top 2 per department (pattern).

```
SELECT * FROM (SELECT e.*, ROW_NUMBER() OVER (PARTITION BY department_id ORDER BY
salary DESC) rn FROM Employees e) t WHERE rn <= 2;
```

32. Advanced Q32: Use ROW_NUMBER to get top 2 per department (pattern).

```
SELECT * FROM (SELECT e.*, ROW_NUMBER() OVER (PARTITION BY department_id ORDER BY
salary DESC) rn FROM Employees e) t WHERE rn <= 2;
```

33. Advanced Q33: Use ROW_NUMBER to get top 2 per department (pattern).

```
SELECT * FROM (SELECT e.*, ROW_NUMBER() OVER (PARTITION BY department_id ORDER BY
salary DESC) rn FROM Employees e) t WHERE rn <= 2;
```

34. Advanced Q34: Use ROW_NUMBER to get top 2 per department (pattern).

```
SELECT * FROM (SELECT e.*, ROW_NUMBER() OVER (PARTITION BY department_id ORDER BY
salary DESC) rn FROM Employees e) t WHERE rn <= 2;
```

35. Advanced Q35: Use ROW_NUMBER to get top 2 per department (pattern).

```
SELECT * FROM (SELECT e.*, ROW_NUMBER() OVER (PARTITION BY department_id ORDER BY
salary DESC) rn FROM Employees e) t WHERE rn <= 2;
```

36. Advanced Q36: Use ROW_NUMBER to get top 2 per department (pattern).

```
SELECT * FROM (SELECT e.*, ROW_NUMBER() OVER (PARTITION BY department_id ORDER BY
salary DESC) rn FROM Employees e) t WHERE rn <= 2;
```

37. Advanced Q37: Use ROW_NUMBER to get top 2 per department (pattern).

```
SELECT * FROM (SELECT e.*, ROW_NUMBER() OVER (PARTITION BY department_id ORDER BY
salary DESC) rn FROM Employees e) t WHERE rn <= 2;
```

38. Advanced Q38: Use ROW_NUMBER to get top 2 per department (pattern).

```
SELECT * FROM (SELECT e.*, ROW_NUMBER() OVER (PARTITION BY department_id ORDER BY
salary DESC) rn FROM Employees e) t WHERE rn <= 2;
```

39. Advanced Q39: Use ROW_NUMBER to get top 2 per department (pattern).

```
SELECT * FROM (SELECT e.*, ROW_NUMBER() OVER (PARTITION BY department_id ORDER BY
salary DESC) rn FROM Employees e) t WHERE rn <= 2;
```

40. Advanced Q40: Use ROW_NUMBER to get top 2 per department (pattern).

```
SELECT * FROM (SELECT e.*, ROW_NUMBER() OVER (PARTITION BY department_id ORDER BY
salary DESC) rn FROM Employees e) t WHERE rn <= 2;
```

41. Advanced Q41: Use ROW_NUMBER to get top 2 per department (pattern).

```
SELECT * FROM (SELECT e.*, ROW_NUMBER() OVER (PARTITION BY department_id ORDER BY
salary DESC) rn FROM Employees e) t WHERE rn <= 2;
```

42. Advanced Q42: Use ROW_NUMBER to get top 2 per department (pattern).

```
SELECT * FROM (SELECT e.*, ROW_NUMBER() OVER (PARTITION BY department_id ORDER BY
salary DESC) rn FROM Employees e) t WHERE rn <= 2;
```

43. Advanced Q43: Use ROW_NUMBER to get top 2 per department (pattern).

```
SELECT * FROM (SELECT e.*, ROW_NUMBER() OVER (PARTITION BY department_id ORDER BY
salary DESC) rn FROM Employees e) t WHERE rn <= 2;
```

44. Advanced Q44: Use ROW_NUMBER to get top 2 per department (pattern).

```
SELECT * FROM (SELECT e.*, ROW_NUMBER() OVER (PARTITION BY department_id ORDER BY
salary DESC) rn FROM Employees e) t WHERE rn <= 2;
```

45. Advanced Q45: Use ROW_NUMBER to get top 2 per department (pattern).

```
SELECT * FROM (SELECT e.*, ROW_NUMBER() OVER (PARTITION BY department_id ORDER BY
salary DESC) rn FROM Employees e) t WHERE rn <= 2;
```

46. Advanced Q46: Use ROW_NUMBER to get top 2 per department (pattern).

```
SELECT * FROM (SELECT e.*, ROW_NUMBER() OVER (PARTITION BY department_id ORDER BY
salary DESC) rn FROM Employees e) t WHERE rn <= 2;
```

47. Advanced Q47: Use ROW_NUMBER to get top 2 per department (pattern).

```
SELECT * FROM (SELECT e.*, ROW_NUMBER() OVER (PARTITION BY department_id ORDER BY
salary DESC) rn FROM Employees e) t WHERE rn <= 2;
```

48. Advanced Q48: Use ROW_NUMBER to get top 2 per department (pattern).

```
SELECT * FROM (SELECT e.*, ROW_NUMBER() OVER (PARTITION BY department_id ORDER BY
salary DESC) rn FROM Employees e) t WHERE rn <= 2;
```

49. Advanced Q49: Use ROW_NUMBER to get top 2 per department (pattern).

```
SELECT * FROM (SELECT e.*, ROW_NUMBER() OVER (PARTITION BY department_id ORDER BY
salary DESC) rn FROM Employees e) t WHERE rn <= 2;
```

50. Advanced Q50: Use ROW_NUMBER to get top 2 per department (pattern).

```
SELECT * FROM (SELECT e.*, ROW_NUMBER() OVER (PARTITION BY department_id ORDER BY
salary DESC) rn FROM Employees e) t WHERE rn <= 2;
```